PaperReader Agent 项目定制面试题 100 问

> 更新时间：2026-04-18
> 目标：这不是泛化 AI Agent 八股，而是严格围绕当前仓库实现整理的项目定制问答。
> 使用方式：先看题目目录，再按模块训练；回答时优先说“目标 -> 设计 -> 代码 -> 风险 -> 效果”。

阅读原则

1. 下面所有回答都以当前仓库代码为准，不以旧审计结论或历史 issue 代替现状。
2. 代码定位优先给出主文件和关键行段，答题时应该能当场打开代码讲。
3. 每道题都尽量补上设计取舍，避免把回答说成“因为作者这么写”。

目录

- 01-10：项目定位与系统边界
- 11-20：API 与任务生命周期
- 21-30：Research Graph 与 LangGraph 编排
- 31-40：Supervisor 与多智能体协作
- 41-50：检索、RAG 与候选论文质量控制
- 51-60：抽取、PaperCard 与上下文压缩
- 61-70：Draft 写作、Skills 与 MCP
- 71-80：Review、Grounding 与置信度
- 81-90：Workspace、持久化与恢复
- 91-100：前端可视化、SSE、测试与后续演进

一、项目定位与系统边界（01-10）

Q01. 这个项目解决的核心问题是什么？

简答：它要把“研究主题 -> 多源论文检索 -> 结构化抽取 -> 压缩整理 -> 英文学术综述生成 -> 引用验证 -> 报告持久化与展示”做成一条可运行的工程链路。
展开：很多 AI Agent Demo 只到“给出一个回答”为止，但这个项目把目标定成科研综述生产系统，所以重点不只是生成，而是检索覆盖、证据压缩、引用可核验、workspace 落盘和前端可追踪。它实际上更像一个 research workflow engine，而不是单轮问答机器人。
代码定位：`AGENTS.md:3-35`，`src/api/routes/tasks.py:45-68,225-251`，`src/research/graph/builder.py:60-124`。
设计取舍 / 风险 / 效果：取舍是链路更长、实现更复杂，但换来的是结果可检查、可复盘、可恢复。风险是任何一个阶段出问题都会拖垮最终质量，因此必须有 review gate 和输出工件。

Q02. 为什么说它不是普通 RAG 问答系统？

简答：因为它不是“检索后拼接上下文回答问题”，而是“检索后构造 paper cards、taxonomy、evidence pools，再生成 survey draft，并继续经过 grounding 与 review”。
展开：普通 RAG 的核心对象是 chunks；这个项目的核心对象是 `paper_cards`、`compression_result`、`draft_report`。中间状态是面向写作任务而设计的，而不是面向问答任务设计的，所以它更接近研究写作流水

线。
代码定位：`src/research/graph/nodes/search.py:117-286`，`src/research/graph/nodes/extract.py:198-372`，`src/research/graph/nodes/extract_compression.py:26-70`，`src/research/graph/nodes/draft.py:142-213`。
设计取舍 / 风险 / 效果：这样做的好处是更适合长文综述；代价是状态更多、调试成本更高。它要求每一层对象都能被序列化、可追踪、能落盘。

Q03. 为什么系统要同时支持 `report workflow` 和 `research workflow`?

简答：因为单篇论文报告和多论文 survey 的输入形态、节点拓扑、输出要求完全不同。
展开：单篇论文报告偏 document ingestion，从 `input_parse -> retrieve_evidence -> draft_report -> resolve_citations -> verify_claims` 走；研究综述偏 topic-driven，从 `clarify -> search_plan -> search -> extract -> extract_compression -> draft -> review` 走。把两者硬塞进一条图会让 state 和条件判断变脏。
代码定位：`src/api/routes/tasks.py:467-511,514-789`，`src/research/graph/builder.py:60-124`，`frontend/src/components/GraphView.tsx:66-120`。
设计取舍 / 风险 / 效果：拆双图增加维护量，但能让节点职责更干净。前端也能按 `source_type` 切不同图谱视图。

Q04. 为什么入口统一放在 `/tasks`，而不是再做一套 CLI 或同步接口?

简答：因为这是一个明显长耗时任务，统一成 task 模型后，前端、数据库、workspace、SSE 都能围绕 task_id 对齐。
展开：同步接口无法自然表达“正在检索/正在写作/正在 review”，而 task 模型可以天然承载状态机、进度事件和中间产物。统一入口还减少了前后端协议分裂。
代码定位：`src/api/routes/tasks.py:23-78,225-251,369-393,514-789`。
设计取舍 / 风险 / 效果：优点是工程一致性更强；缺点是任务管理、快照和清理逻辑要自己维护。这个项目当前已经把 task 做成了主运行面，而不是旁路。

Q05. 为什么任务执行必须异步化?

简答：因为检索、全文下载、LLM 抽取、写作、review 都是高延迟操作，同步阻塞会让接口超时并破坏交互体验。
展开：`create_task` 只负责落 task record 并启动后台协程，真正的图运行在 `_run_graph()` 里完成。这样用户拿到 task_id 后，可以用 `/events` 订阅进度，再用 `/result` 拉最终结果。
代码定位：`src/api/routes/tasks.py:225-251,369-393,514-789`。
设计取舍 / 风险 / 效果：异步让前端体验更像一个运行中的 agent，而不是一个长轮询接口；代价是要处理幂等、失败回写和进度一致性。

Q06. 为什么系统同时保留结构化状态和 Markdown 报告?

简答：因为前者服务于图执行、review、恢复和调试，后者服务于用户消费、SSE 预览和最终交付。
展开：例如 `draft_report`、`review_feedback`、`rag_result` 都是结构化对象；`draft.md` 和 `report.md` 才是给人看的最终文本。只有结构化状态，前端不好展示；只有 Markdown，又很难做 claim verification 和恢复。
代码定位：`src/api/routes/tasks.py:110-145,278-316`，`src/agent/output_workspace.py:190-282,285-343`。

设计取舍 / 风险 / 效果：双轨数据会增加序列化成本，但显著提升了可观察性和可调试性。

Q07. 为什么项目既有 PostgreSQL 持久化，又有 `output/workspaces/` 文件产物？

简答：两者用途不同，数据库负责 durable snapshot 和 report record，workspace 负责人类可读的工件与回放体验。
展开：数据库更适合存 task snapshot、最终 report、接口恢复；workspace 更适合存 `brief.json`、`draft.md`、`review_feedback.json` 这类中间工件。前端 SSE 甚至直接监听 workspace 文件变化，做 live preview。
代码定位：`AGENTS.md:18-24`，`src/api/routes/tasks.py:15-21,91-107,157-220,760-788`，`src/agent/output_workspace.py:1-26,118-162,285-343`。
设计取舍 / 风险 / 效果：数据库保证 durability，workspace 保证可见性。缺点是需要维护两套一致性，但收益很高。

Q08. `AGENTS.md` 里的硬约束对系统架构影响最大的是哪几条？

简答：最关键的是 PostgreSQL only、LangGraph official API only、以及任务结果接口对齐。
展开：它直接禁止了用 SQLite 偷懒做元数据持久化，也禁止手写 for-loop 模拟 agent 图执行。它逼着系统必须在 `StateGraph`、checkpointer、PostgreSQL snapshot 这些正式工程方案上收敛。
代码定位：`AGENTS.md:16-35`。
设计取舍 / 风险 / 效果：这类约束牺牲了一些“快速糊出来”的速度，但把架构拉回到了长期可维护的方向。

Q09. 为什么项目明确禁止 SQLite？

简答：因为项目已经把“长生命周期持久化”定义成 PostgreSQL 的职责，SQLite 会破坏环境一致性、并发语义和部署路径。
展开：task state、report record、测试基座如果混入 SQLite，会出现开发环境能跑、生产环境语义不同的问题。对一个本来就要处理并发任务、恢复和 snapshot 的系统来说，这种双存储分裂非常危险。
代码定位：`AGENTS.md:18-24`。
设计取舍 / 风险 / 效果：代价是本地启动稍重，但换来的是测试、开发、部署路径一致，不会埋下“本地 SQLite 假稳定”的坑。

Q10. 如果一句话概括这个系统，最准确的说法是什么？

简答：它是一个面向科研综述生成的、基于 LangGraph 的多阶段多智能体研究工作流系统。
展开：关键词必须同时覆盖“科研综述”“多阶段”“LangGraph”“多智能体”“review/grounding”“workspace artifacts”。少任何一个词，都会把项目讲扁，要么讲成普通 RAG，要么讲成普通聊天机器人。
代码定位：`AGENTS.md:3-14`，`src/research/graph/builder.py:60-124`，`src/research/agents/supervisor.py:1-129`。
设计取舍 / 风险 / 效果：这种概括方式更接近真实实现，也更容易接住面试官后续追问。

二、API 与任务生命周期（11-20）

Q11. `create_task` 在系统里真正做了哪些事？

简答：校验输入、标准化 `source_type`、创建 `TaskRecord`、分配 `workspace_id`、落 snapshot，并异步启动 `_run_graph`。
展开：它没有直接做任何检索或生成；它只是把“任务被接受”这件事原子化。这样即使后面的 graph 失败了，也有可恢复的 task 元数据。
代码定位：`src/api/routes/tasks.py:225-251`。
设计取舍 / 风险 / 效果：这样设计能把“提交任务”和“执行任务”解耦，降低前端等待时间。代价是要维护任务生命周期状态。

Q12. 为什么 `workspace_id` 在创建任务时就要生成，而不是跑完再建?

简答：因为中间产物要一边跑一边写，而且 SSE 需要尽早知道自己应该监听哪个 workspace。
展开：如果只在结束时创建 workspace，那么 `brief.json`、`draft.md`、`review_feedback.json` 都没法有稳定归属，也无法把多个 task 归到同一个 workspace 里。它本质上是“会话容器”，不是最终结果容器。
代码定位：`src/api/routes/tasks.py:227-239,527-541`，`src/agent/output_workspace.py:49-99,118-162`。
设计取舍 / 风险 / 效果：提前分配 workspace 能增强恢复与组织能力，但需要处理“建了 workspace 但任务失败”的情况。

Q13. `/tasks/{id}`、`/tasks/{id}/result` 为什么要保持行为对齐?

简答：因为前者是状态视图，后者是结果视图，两者一旦不对齐，前端就会出现“列表说完成了、结果却为空”这种体验错误。
展开：`AGENTS.md` 已经把这条写成硬规则。当前实现中，`_task_payload` 和 `get_task_result` 都会回传 report、brief、search_plan、rag_result、paper_cards 等关键字段，只是侧重点不同。
代码定位：`AGENTS.md:23-25`，`src/api/routes/tasks.py:110-145,278-316`。
设计取舍 / 风险 / 效果：重复字段看似啰嗦，但它减少了 API 语义不一致导致的前端补丁。

Q14. 为什么 `get_task_result` 要同时返回 `result_markdown`、`draft_markdown`、`full_markdown`?

简答：因为系统支持 research/report 双工作流，且 report workflow 还有 `draft/full` 两种模式。
展开：不是所有任务都会有 `full_markdown`，也不是所有 research task 都会先得到一个“规范化 final report”。把多个字段同时暴露出来，可以让前端和测试更清楚地知道当前拿到的到底是哪一版文本。
代码定位：`src/api/routes/tasks.py:278-316`。
设计取舍 / 风险 / 效果：字段会显得多，但它避免了客户端猜测“当前 markdown 属于哪种模式”。

Q15. 为什么 `/tasks/{id}/chat` 只允许已完成任务进入?

简答：因为这个 chat 当前不是 workflow steering，而是基于已生成报告的后续问答。
展开：接口先检查 `task.status == completed`，再读取 `report_context_snapshot or result_markdown`。这说明当前 chat 是“围绕最终报告继续聊”，而不是“中途控制 agent 去改变计划”。
代码定位：`src/api/routes/tasks.py:319-366`。
设计取舍 / 风险 / 效果：这样简单且稳定，但也暴露出当前系统对“运行中交互”和“post-report 主动 skills”支持不足。

Q16. `report_context_snapshot` 的设计意义是什么?

简答：它把任务产出的最终可聊上下文冻结下来，避免 chat 再去读取一堆零散状态拼上下文。
展开：对 research task，`result_markdown` 会被写回 `report_context_snapshot`；对 report workflow，如果是 full mode 则优先 full markdown。这样 chat 层不需要理解复杂工作流，只需要围绕一个稳定文本继续问答。
代码定位：`src/api/routes/tasks.py:327-345,645-754`。
设计取舍 / 风险 / 效果：简化了 chat 层，但也意味着当前 chat 对结构化中间工件利用不够深。

Q17. `/tasks/{id}/events` 的 SSE 事件流是怎么工作的?

简答：它同时流两类事件，一类来自内存中的 `task.node_events`，一类来自 workspace 文件变化。
展开：前者负责 `node_start`、`node_end`、`status_change`；后者负责 `artifact_ready` 和 Markdown `report_snapshot`。因此用户能同时看到“节点推进”和“文稿真的在变”。
代码定位：`src/api/routes/tasks.py:157-220,369-393`。
设计取舍 / 风险 / 效果：好处是可视化信息很丰富；风险是事件流来源有两套，需要注意去重与顺序感。

Q18. `_run_graph` 是怎么在 research 和 report 两条链路之间分发的?

简答：它先看 `source_type`，`research` 走 supervisor 驱动的 research graph，其他走 report graph。
展开：research 路径会创建 supervisor、构造 research state、选择是否 handoff；report 路径则直接编译 report graph，并在线程池里跑 `_run_graph_sync`。两条路最终都会回填 `TaskRecord` 并统一落库、落 workspace。
代码定位：`src/api/routes/tasks.py:514-789`。
设计取舍 / 风险 / 效果：统一入口、内部路由的方式更利于前端与 DB 统一；代价是 `_run_graph` 变成了很关键的汇聚层。

Q19. 为什么要用 `_build_state_template` 统一 state 初始字段?

简答：因为 report 和 research 两条图虽然不同，但共享大量状态键，模板化能减少遗漏和前后不一致。
展开：模板里已经预留了 `brief`、`search_plan`、`paper_cards`、`compression_result`、`draft_report`、`review_feedback`、`node_statuses` 等字段。这既方便图节点读写，也方便最后统一回填到 task record。
代码定位：`src/api/routes/tasks.py:467-511`。
设计取舍 / 风险 / 效果：统一模板能减少状态飘移，但也要警惕模板持续膨胀成“万能 dict”。

Q20. 为什么系统同时保留内存 `_tasks` 和数据库 snapshot?

简答：内存用于当前进程的低延迟读写，数据库用于 durable 恢复。
展开：`_get_task_record()` 会先查内存，没找到再从 `load_task_snapshot()` 恢复。这样列表页和详情页可以跨进程重启后继续看到旧任务，而运行中的状态更新又不用每次都从数据库反序列化。
代码定位：`src/api/routes/tasks.py:25,91-107,254-267`。
设计取舍 / 风险 / 效果：这是典型的 cache + source of truth 组合。风险是两边同步失败时会出现短暂不一致，因此 `_sync_task_snapshot()` 很关键。

三、Research Graph 与 LangGraph 编排（21-30）

Q21. Research workflow 是如何满足 LangGraph 规范的?

简答：核心是直接使用 `StateGraph` 声明节点和边，而不是手写 Python 循环模拟工作流。
展开：`build_research_graph()` 明确把节点、条件边和终止条件全写在图里，并可选挂 checkpointer。这是符合 `AGENTS.md` 里 LangGraph 约束的实现方式。
代码定位：`AGENTS.md:27-35`，`src/research/graph/builder.py:60-124`。
设计取舍 / 风险 / 效果：图式声明的好处是流程清晰、可追踪、可插入检查点；代价是新增节点时需要同时维护 state 和图拓扑。

Q22. 当前 research graph 的 8 个节点分别是什么?

简答：`clarify -> search_plan -> search -> extract -> extract_compression -> draft -> review -> persist_artifacts`。
展开：这条链路对应一个标准的 research production pipeline：先把题目说清楚，再规划检索，再拿论文，再抽卡片，再做压缩，再写草稿，再审稿，最后把结果整理成可消费工件。
代码定位：`src/research/graph/builder.py:78-120`。
设计取舍 / 风险 / 效果：节点够清晰，所以每个阶段都容易被单测和观测；但链路变长后，失败传播必须靠 review 和错误回写兜底。

Q23. `clarify` 之后为什么可能直接结束?

简答：因为 `clarify` 的职责是把原始 topic 变成结构化 brief，如果 brief 不成立或者需要 follow-up，就不应该盲目往下跑。
展开：`_route_after_clarify()` 只有在 `brief` 存在且 `awaiting_followup` 为假时才进入 `search_plan`。这避免了检索系统在输入不清晰时制造大量离题文献。
代码定位：`src/research/graph/builder.py:26-33,101-106`。
设计取舍 / 风险 / 效果：早停会增加一次交互成本，但它能显著减少下游 off-topic 污染。

Q24. `search_plan` 之后为什么还要看 `research_depth`?

简答：因为系统既支持 plan-only 模式，也支持 full research 模式。
展开：`_route_after_search_plan()` 明确要求 `research_depth == full` 才进入检索，否则停在规划阶段。这种模式很适合测试、交互澄清或只想先看问题拆解的场景。
代码定位：`src/research/graph/builder.py:36-42,108-111`。
设计取舍 / 风险 / 效果：增加模式分支会复杂一点，但给系统带来了非常有用的“轻量规划模式”。

Q25. 为什么 `review_passed` 才能进入 `persist_artifacts`?

简答：因为系统不想把未通过审查的报告包装成“正式完成”的最终产物。
展开：`_route_after_review()` 直接以 `review_passed` 为 gate。即使 review 不通过，task state 仍然保留，用户也能看到 draft 和 revisions，但不会被当成正式 report。
代码定位：`src/research/graph/builder.py:45-47,116-119`。
设计取舍 / 风险 / 效果：这种设计让 review 真正具备约束力，而不是走过场。副作用是用户可能更常见到“有内容但未通过”的中间状态。

Q26. `current_stage` 为什么要作为通用状态字段持续写回?

简答：因为前端、API 和调试日志都需要一个统一的“现在跑到哪一步了”的信号。
展开：`_with_current_stage()` 会在节点输出里自动补 `current_stage`。这样无论后端日志、详情接口还是 SSE 状态展示，都可以依赖同一个字段，而不是各写各的。
代码定位：`src/research/graph/builder.py:50-57,80-98`，`src/api/routes/tasks.py:628-643`。
设计取舍 / 风险 / 效果：这是一个很小但很关键的可观测性设计，避免了 UI 和后端对阶段状态理解不一致。

Q27. Research graph 是怎么接入 checkpoint 的?

简答：通过 `use_checkpointer` 参数，在编译图时挂上 `get_langgraph_checkpointer("research_graph")`。
展开：checkpointer 的真正后端由环境变量决定，默认是 `MemorySaver`，也可以切到 `PostgresSaver`。这符合 LangGraph 约束，也为未来做持久恢复打好了接口。
代码定位：`src/research/graph/builder.py:60-64,122-124`，`src/agent/checkpointing.py:22-68`。
设计取舍 / 风险 / 效果：当前默认 memory 更适合本地与测试，但生产如果要真正断点恢复，最好切到 postgres backend。

Q28. 为什么这条图被称为“阶段显式、节点内可并行”，而不是任意 DAG?

简答：因为图本身几乎是线性的，并行性主要藏在节点内部，例如 search 的多源并发和 extract 的批量抽取。
展开：这和很多宣传图里“复杂 fan-out / fan-in DAG”不同。当前设计更注重工程确定性，减少跨节点并发带来的状态合并复杂度。
代码定位：`src/research/graph/builder.py:101-120`，`src/research/graph/nodes/search.py:38-111,165-173`，`src/research/graph/nodes/extract.py:198-372`。
设计取舍 / 风险 / 效果：阶段线性让系统更稳，但吞吐上限比完全并行 DAG 低一些。

Q29. 为什么把 `extract_compression` 明确做成独立节点，而不是塞进 `draft` 里?

简答：因为压缩本身就是一个独立的、可复用的中间加工阶段，它有自己的输入、输出和观测价值。
展开：如果把它写进 `draft`，那就很难单独评估压缩效果，也难以把 `taxonomy`、`compressed_cards`、`evidence_pools` 当成中间工件落盘。独立节点让它在架构上成为“一等公民”。
代码定位：`src/research/graph/builder.py:81-89,113-114`，`src/research/graph/nodes/extract_compression.py:1-77`。
设计取舍 / 风险 / 效果：多一个节点会让图更长，但压缩是否有效终于可以被观察和单测。

Q30. 如果要新增一个节点，最安全的改法是什么?

简答：先判断是否已有官方 LangGraph / LangChain API 能覆盖，再改 state template、graph builder、workspace 输出和前端图谱。
展开：只改 builder 不够，因为 `tasks.py`、`GraphView.tsx`、SSE 文件流、`output_workspace.py` 可能都要同步。真正危险的不是“加一个函数”，而是节点拓扑变化后 UI 和产物没有跟上。
代码定位：`AGENTS.md:34-35`，`src/research/graph/builder.py:60-124`，`src/agent/output_workspace.py:285-343`，`frontend/src/components/GraphView.tsx:66-120`。
设计取舍 / 风险 / 效果：流程层面的改动一定要跨后端、存储、前端一起看，否则最容易出现 graph drift。

四、Supervisor 与多智能体协作（31-40）

Q31. 为什么可以说当前 supervisor 已经比历史版本更合规?

简答：因为它已经明确使用官方 `create_supervisor` 和 `create_react_agent`，不再把核心编排建立在手写 for-loop 上。
展开：虽然 `AgentSupervisor` 这个 facade 还在，但模块头部已经写明“public facade delegates orchestration to official supervisor API”。这说明它保留了现有接口形状，但底层思路已经转向官方模式。
代码定位：`src/research/agents/supervisor.py:1-29,221-255`，`AGENTS.md:31-35`。
设计取舍 / 风险 / 效果：保留 facade 减少了对 API 层的冲击，但也意味着仓库里还会保留一些过渡痕迹。

Q32. `create_react_agent` 在这里扮演什么角色?

简答：它负责把单个 worker agent 包装成 LangGraph/LangChain 官方可管理的执行单元。
展开：这个项目并不是自己手写 tool loop，而是用官方 prebuilt agent 作为 worker 的构建基座，再由 supervisor 决定协作顺序。这样更容易和官方消息、tool、handoff 机制兼容。
代码定位：`src/research/agents/supervisor.py:19-25`。
设计取舍 / 风险 / 效果：使用 prebuilt agent 降低了自定义灵活度，但显著减少了“半自定义半手写”的复杂度。

Q33. 为什么还要保留 `AgentSupervisor` 这个外观类?

简答：因为 API 层和其它模块已经围绕这个 facade 组织，直接删掉会造成更大的接口震荡。
展开：它现在更像是一个 adapter：对外仍然暴露 `collaborate`、`run_node` 之类的接口，对内则根据配置切换 official supervisor、legacy backend、v2 backend。换句话说，它是在给架构迁移提供缓冲层。
代码定位：`src/research/agents/supervisor.py:221-255`，`src/api/routes/tasks.py:548-581`。
设计取舍 / 风险 / 效果：好处是兼容已有调用方；坏处是 facade 容易继续膨胀，需要警惕重新长成“新旧逻辑都混进去”的上帝类。

Q34. `CANONICAL_NODE_ORDER` 的作用是什么?

简答：它给 supervisor 一个明确的阶段语义，不让 agent 协作变成完全自由的黑盒对话。
展开：当前多 agent 不是“所有 agent 任意互聊”，而是围绕固定阶段顺序协作：澄清、规划、检索、抽取、压缩、写作、评审、持久化。这个顺序和 research graph 本身保持一致。
代码定位：`src/research/agents/supervisor.py:86-95`，`src/research/graph/builder.py:78-120`。
设计取舍 / 风险 / 效果：这种方式更适合工程交付，虽然不如完全自治式系统“看起来智能”，但更可靠。

Q35. 什么是 legacy backend 和 v2 backend 切换?

简答：就是同一个阶段节点可以按配置走旧实现或新 agent 化实现。
展开：`LEGACY_NODE_TARGETS` 映射到 graph node 函数，`V2_AGENT_TARGETS` 映射到 planner/retriever/analyst/reviewer agent。`_get_backend_mode()` 会根据全局执行模式和节点配置做决定。
代码定位：`src/research/agents/supervisor.py:97-129,239-255`。
设计取舍 / 风险 / 效果：这是一个典型的迁移期架构手段，方便灰度；缺点是回答架构时必须讲清“当前主路是什么，兼容层是什么”。

Q36. 为什么要允许 `register_backend()` 做节点级 override?

简答：因为某些环境下需要对单节点做实验替换、测试替换或策略切换。
展开：有了节点级 backend override，可以在不改整条流程的前提下替换 search、draft、review 的实现。这对回归测试、灰度迁移和问题隔离都很有价值。
代码定位：`src/research/agents/supervisor.py:224-245,257-260`。
设计取舍 / 风险 / 效果：灵活度更高，但如果 override 太多，也会让系统行为难以解释，所以应当控制使用范围。

Q37. 为什么 supervisor 还要主动把节点结果同步到 workspace?

简答：因为多 agent 路径下，光有内存 state 不够，用户和调试者需要看到文件级中间产物。
展开：`_sync_node_to_workspace()` 会调用 `write_node_output()`，把各节点结果落成对应文件。这样无论节点是 legacy graph node 还是 v2 agent，都能保持相似的产物可见性。
代码定位：`src/research/agents/supervisor.py:42-55`，`src/agent/output_workspace.py:285-343`。
设计取舍 / 风险 / 效果：这让 supervisor 不只是编排器，也是产物同步桥。好处是用户体验一致，代价是 supervisor 多承担了一点 I/O 责任。

Q38. Review 失败时为什么要追加 revision 文件?

简答：因为用户需要看到“哪一稿被挡住了”，而不是只得到一个 pass/fail 布尔值。
展开：`_append_review_revision()` 会把当前 draft 以 `after_review` 之类的标签写进 `revisions/`。这让系统保留了失败现场，后续人工复盘和自动修复都更容易做。
代码定位：`src/research/agents/supervisor.py:57-83`，`src/agent/output_workspace.py:227-258`。
设计取舍 / 风险 / 效果：revision 会增加输出文件数量，但它极大提升了调试和解释能力。

Q39. `SupervisorMode` 和 `use_handoff` 的关系是什么?

简答：是否启用 handoff 不是硬编码，而是由 supervisor config 决定。
展开：`tasks.py` 在 research 路径里读取 `supervisor.config.supervisor_mode`，如果是 `LLM_HANDOFF` 才会把 `use_handoff=True` 传给 `_run_supervisor_sync()`。这说明 handoff 是一个可切换策略，而不是默认唯一实现。
代码定位：`src/api/routes/tasks.py:548-581`。
设计取舍 / 风险 / 效果：策略切换更灵活，但也意味着评测时要明确当前运行模式，否则结果不可比。

Q40. 怎么诚实地描述这个项目的“多智能体”程度?

简答：它是“官方 supervisor + 分阶段 worker + 可切换 backend”的工程化多 agent 系统，不是完全自治式 agent society。
展开：各 worker 的职责还是围绕既定阶段展开，supervisor 也有明确的 canonical order，所以它的价值在于责任分层和实现隔离，而不是让所有 agent 自发涌现复杂协作。这个描述既准确，也更容易获得工程面试官认可。
代码定位：`src/research/agents/supervisor.py:1-29,86-129,221-255`。
设计取舍 / 风险 / 效果：少吹“自治”，多讲“工程确定性”，反而更强。

五、检索、RAG 与候选论文质量控制（41-50）

Q41. `search_node` 的输入和输出是什么?

简答：输入是 `search_plan` 和 `brief`，输出是带 `paper_candidates`、trace、coverage notes 的 `RagResult`。
展开：它不仅返回论文列表，还记录 `retrieval_trace`、`dedup_log`、`rerank_log`、正文下载统计等信息。也就是说，这个节点既是 retrieval 节点，也是检索审计节点。
代码定位：`src/research/graph/nodes/search.py:117-286`。
设计取舍 / 风险 / 效果：输出更重，但后续 review 和评测才有足够的检索链路证据。

Q42. 为什么检索要并行走 SearXNG、arXiv API、DeepXiv 三路?

简答：因为三者擅长的点不同，SearXNG 负责广度、arXiv API 负责精度元数据、DeepXiv 负责 TLDR 和趋势补充。
展开：代码里甚至直接把这三条策略写在模块注释中。最终检索目标不是“找最多”，而是兼顾 coverage、metadata 完整性和后续抽取可用性。
代码定位：`src/research/graph/nodes/search.py:1-9,156-180,256-280`。
设计取舍 / 风险 / 效果：三源并发提高召回质量，但也引入了去重和优先级排序问题。

Q43. 为什么没有时间范围时会自动加默认年份过滤?

简答：为了避免主题搜索时抓到大量过旧论文，稀释当前 survey 的相关性。
展开：`search_node` 会把空时间范围回退到默认年份过滤。这个细节很工程化，因为很多用户不会主动写时间范围，但 survey 对时效性通常非常敏感。
代码定位：`src/research/graph/nodes/search.py:156-164`。
设计取舍 / 风险 / 效果：好处是减轻旧论文污染；风险是可能漏掉真正的经典起源论文，所以最好在 brief 里允许显式放宽时间窗。

Q44. 为什么去重优先级是 `arXiv API > DeepXiv > SearXNG`?

简答：因为 metadata 完整性依次下降，先保留高质量元数据版本更利于后续抽取。
展开：代码在合并阶段已经把这个优先级写死。一个同样标题的候选，如果 arXiv API 已经给了完整 `arxiv_id`、`authors`、`url`，就没必要再让低质量搜索结果覆盖它。
代码定位：`src/research/graph/nodes/search.py:179-231`。
设计取舍 / 风险 / 效果：优点是后续 paper card 更稳；风险是 DeepXiv 的某些补充字段可能被忽略，但后面还有 enrich 和 merge 过程。

Q45. `strict-core rerank` 解决的核心问题是什么?

简答：解决“召回很多，但主题核心意图严重跑偏”的问题。
展开：系统定义了严格主题锚点和 fatal penalties，用于把 off-topic 的候选再次筛掉。它本质上是在 search 阶段提前做一次“轻量 topic QA”，防止离题论文污染全文下载、抽取和写作。
代码定位：`src/research/graph/nodes/search.py:24-35,233-238`。
设计取舍 / 风险 / 效果：会牺牲一部分召回，但能显著降低 `off_topic_ratio`。

Q46. 为什么检索后要立刻把候选论文 ingest 到本地语料层?

简答：因为后续 extract、draft、grounding 都更需要稳定的本地证据，而不是每次重新访问外部源。
展开：`_ingest_paper_candidates()` 会优先尝试正文下载和 chunking，失败时再退到 abstract。它还会把正文证据回填进 candidate，后续抽取时能优先用 fulltext snippets。
代码定位：`src/research/graph/nodes/search.py:240-276,289-320`。
设计取舍 / 风险 / 效果：多了一步 ingest，但后续阶段的 evidence 质量和稳定性都会更好。

Q47. `fulltext_ratio` 为什么是一个关键指标?

简答：因为它直接决定了系统是在“读正文写综述”，还是“只看摘要硬写”。
展开：代码会记录全文尝试数、成功数和比率，并写进 `coverage_notes`。如果这个值低，后续 report 即使可读，grounding 和 citation quality 也往往会明显下降。
代码定位：`src/research/graph/nodes/search.py:250-275`。
设计取舍 / 风险 / 效果：这个指标非常适合作为面试里解释质量差异的抓手，因为它比“模型变笨了”更具体。

Q48. 为什么 `retrieval_trace`、`dedup_log`、`coverage_notes` 很重要?

简答：它们让检索不再是黑盒，可以解释为什么只找到这些论文。
展开：真实 AI Agent 项目最容易被质疑的就是“你怎么知道它没漏 / 没跑偏”。这些 trace 字段就是给复盘、评测、review 和前端调试准备的审计材料。
代码定位：`src/research/graph/nodes/search.py:256-280`。
设计取舍 / 风险 / 效果：字段更复杂，但可解释性显著增强。

Q49. 当 `search_plan` 为空或者 query 为空时，系统怎么处理?

简答：直接返回 `rag_result=None` 或跳过，而不是硬造结果。
展开：`search_node` 在没有 `search_plan` 或没有有效 `all_queries` 时都会提前返回，并写 warning。这个策略是对的，因为虚构检索结果只会把错误埋到更后面。
代码定位：`src/research/graph/nodes/search.py:132-155`。
设计取舍 / 风险 / 效果：早失败看起来“不智能”，但比错误传播到写作阶段强得多。

Q50. 如果面试官问“为什么 paper_count 会从一轮到另一轮掉很多”，你怎么答?

简答：通常不是模型随机波动，而是 planner query、strict rerank、fulltext ingest 或时间过滤在起作用。
展开：这个系统里 paper_count 是多个阶段共同作用的结果，不是 search API 原样返回值。尤其当 strict-core rerank 变严、默认年份过滤生效、planner fallback 变窄时，paper_count 下降很正常。
代码定位：`src/research/graph/nodes/search.py:139-173,233-280`。
设计取舍 / 风险 / 效果：解释 paper_count 时一定要同时讲“数量”和“质量门”，否则很容易陷入“多就是好”的误区。

六、抽取、PaperCard 与上下文压缩（51-60）

Q51. `extract` 阶段的核心设计思想是什么?

简答：把原始候选论文转成结构化 `PaperCard`，并优先保留真实证据而不是只保留标题和摘要。
展开：它不只是调用一次 LLM，总体策略是 Progressive Reading：先拿 DeepXiv brief，再做批量结构化抽取，再做 fallback。这说明系统把“卡片质量”当成写作质量的前置条件。
代码定位：`src/research/graph/nodes/extract.py:198-372`。
设计取舍 / 风险 / 效果：多层抽取链路更复杂，但比单步 LLM 摘要稳得多。

Q52. DeepXiv brief 在抽取阶段的真实作用是什么?

简答：它提供 TLDR、keywords、GitHub URL 等轻量但高价值的补充证据。
展开：代码会先并行抓取所有带 arXiv ID 的 brief，再在抽取和 fallback 阶段合并进 card。这样即使全文下载失败，系统也不至于退化成只靠原始 abstract。
代码定位：`src/research/graph/nodes/extract.py:215-233,430-469`。
设计取舍 / 风险 / 效果：DeepXiv 提升了抽取鲁棒性，但也引入了额外外部依赖，需要在面试里说明它是“增强项”而不是唯一证据源。

Q53. 为什么批量抽取时把 `BATCH_SIZE` 设成 3?

简答：是为了控制 token 体积和超时风险，而不是为了数学上最优。
展开：代码注释已经写明“每批 3 篇，减少 token 数量，避免超时”。这类参数背后反映的是工程经验：抽取质量和请求稳定性之间要平衡。
代码定位：`src/research/graph/nodes/extract.py:235-270,321-326`。
设计取舍 / 风险 / 效果：批太大容易超时或 JSON 崩；批太小吞吐太差。3 是一个保守稳妥值。

Q54. 抽取时为什么强调 `fulltext > DeepXiv TLDR > abstract`?

简答：因为后续写综述和做 grounding，最需要的是正文级证据。
展开：代码在构造每篇 paper 输入时会优先取 `fulltext_snippets`，没有正文才退到 DeepXiv，再退到 abstract。这不是简单的“信息越多越好”，而是证据强度的排序。
代码定位：`src/research/graph/nodes/extract.py:286-314`。
设计取舍 / 风险 / 效果：证据层次更清楚，能解释为什么 `fulltext_ratio` 上来后 report 质量通常会提高。

Q55. `_extract_json()` 为什么值得单独讲?

简答：因为结构化抽取失败最常见的不是模型不会，而是返回 JSON 不规范。
展开：它会去掉 code fence，尝试数组和对象两种边界，再做 `json.loads` 验证。这类“输出恢复层”在真实 LLM 系统里很关键，否则上游一次格式波动就会把整批抽取打穿。
代码定位：`src/research/graph/nodes/extract.py:472-517`。
设计取舍 / 风险 / 效果：多一道恢复逻辑会增加代码复杂度，但能显著提升批量抽取的稳定性。

Q56. 为什么 `_simple_card()` 这样的 fallback 很重要?

简答：因为即使 LLM 抽取失败，系统也必须保住最基本的 evidence，不能让后续 grounding 完全失明。
展开：`_simple_card()` 明确把原始 abstract 或全文证据保留下来，并在注释里写出它是后续 `ground_draft_report` 和 `verify_claims` 的主要 evidence 来源。这个思路非常工程化。
代码定位：`src/research/graph/nodes/extract.py:660-734`。
设计取舍 / 风险 / 效果：fallback card 信息不够漂亮，但能让链路继续跑，并为 review 提供最小可用证据。

Q57. 为什么还要在抽取后做 `_enrich_card()` 回填?

简答：因为 LLM 最容易丢失标题、作者、URL、arXiv ID 这些“看起来简单但后果严重”的字段。
展开：代码会用原始 candidate 回填 title/authors/url/arxiv_id，还会把 fulltext snippets 和 evidence_text 回写进去。它本质上是在做“防 LLM 信息蒸发”。
代码定位：`src/research/graph/nodes/extract.py:534-639`。
设计取舍 / 风险 / 效果：这一步提高了 card 的 metadata 稳定性，对 citation resolution 和后续下载都非常关键。

Q58. 为什么 `methods`、`datasets` 还要用规则词表补提取?

简答：因为某些结构化字段让 LLM自由生成会不稳定，而 survey 写作又非常依赖这些横向维度。
展开：`_extract_entities()` 配合一长串关键词，能把一些常见 benchmark、方法名快速扫出来。这不是为了取代 LLM，而是为了补齐横向比较需要的结构。
代码定位：`src/research/graph/nodes/extract.py:542-576,690-715`。
设计取舍 / 风险 / 效果：规则法不优雅，但对比较矩阵和评测章节很有帮助。

Q59. `extract_compression_node` 的输出到底是什么?

简答：主要是 `compression_result` 和 `taxonomy`，其中 `compression_result` 包含 compressed cards、taxonomy 和 evidence pools。
展开：节点把原始 `paper_cards` 压成更适合写作的中间对象，并把 taxonomy 作为单独字段返回，方便后续 draft 和展示使用。日志里还会打印压缩比例。
代码定位：`src/research/graph/nodes/extract_compression.py:26-70`。
设计取舍 / 风险 / 效果：中间对象更多了，但写作阶段不再需要把所有原始 card 原样塞进上下文。

Q60. 上下文压缩真正解决了什么问题?

简答：解决的是“论文多了以后，draft 只能靠截断前 N 篇卡片写作”的问题。
展开：压缩后，系统不再只看卡片堆，而是拿到 taxonomy、compressed claims、section evidence pools 这类更适合长文写作的表示。它把“证据仓库”变成了“可写作素材”。
代码定位：`src/research/graph/nodes/extract_compression.py:34-40`，`src/research/graph/nodes/draft.py:147-185,219-263`。
设计取舍 / 风险 / 效果：压缩本身可能损失细节，但如果不做压缩，大规模 survey 几乎必然被 context window 压垮。

七、Draft 写作、Skills 与 MCP（61-70）

Q61. `draft_node` 除了“写报告”之外还做了什么?

简答：它还负责运行写作辅助 skill、写 scaffold preview、流式发布 markdown snapshot，并把 skill 工件写回 state。
展开：这说明 draft 阶段不是一次黑箱 LLM 生成，而是一个组合阶段：先生成比较矩阵和 scaffold，再调用主写作模型，再把中间成果写入 workspace 与 SSE。
代码定位：`src/research/graph/nodes/draft.py:41-139,142-213`。
设计取舍 / 风险 / 效果：阶段更复杂，但显著提升了写作可控性和可观察性。

Q62. `build_drafting_skill_artifacts()` 的作用是什么?

简答：它把写作前需要的结构化辅助材料一次性准备好，避免主模型裸写。
展开：当前依次触发 `comparison_matrix_builder`、`writing_scaffold_generator`、`academic_review_writer_prompt` 三类能力，并把结果打包成 `skill_artifacts`。这相当于给写作模型准备了一份“结构化备考资料”。
代码定位：`src/research/graph/nodes/draft.py:41-139`。
设计取舍 / 风险 / 效果：前置准备会增加时延，但能显著改善章节组织和证据分布。

Q63. 为什么写作前先做 `comparison_matrix_builder`?

简答：因为 survey 最怕变成一篇篇 paper 的摘要拼接，而比较矩阵天然要求横向比较。
展开：矩阵把 methods、datasets、benchmarks、limitations 拉成统一视角，这会强迫后续写作从“横向结构”出发，而不是从“顺着论文编号写”出发。
代码定位：`src/research/graph/nodes/draft.py:96-105`，`src/skills/registry.py:467-478`。
设计取舍 / 风险 / 效果：矩阵可能不够细，但它是防止综述写成流水账的关键结构。

Q64. `writing_scaffold_generator` 解决了什么问题?

简答：它解决的是“大纲松散、章节没组织逻辑、每节不知道该拿哪些论文支撑”的问题。
展开：这个 skill 会生成 scaffold、outline、section evidence map 和 writing guidance。也就是说，它不是只给标题，而是在给主模型一个章节级的写作脚手架。
代码定位：`src/research/graph/nodes/draft.py:109-123`，`src/skills/registry.py:493-504`。
设计取舍 / 风险 / 效果：脚手架提高了组织性，但如果 scaffold 太死，也可能压制一些更好的生成自由度。

Q65. `academic_review_writer_prompt` 为什么通过 MCP 接入?

简答：因为它被定义为一个 prompt/resource 型外部能力，而不是普通本地函数。
展开：draft 阶段通过 skill registry 调用 `academic_review_writer_prompt`，底层走 `MCP_PROMPT` backend，再由 `mcp_adapter` 调用 server 的 `prompts/get`。这说明项目已经在把写作规范能力外置成可替换资源。
代码定位：`src/research/graph/nodes/draft.py:126-137`，`src/skills/registry.py:506-525`，`src/tools/mcp_adapter.py:229-260,283-320`。
设计取舍 / 风险 / 效果：MCP 让能力来源更模块化，但也增加了 server discovery 和 transport 兼容的复杂度。

Q66. 为什么 `draft.py` 里会显式定义 section order 和 citation floors?

简答：因为 survey 不是自由散文，它有明确的章节职责和最低引用覆盖要求。
展开：`SURVEY_SECTION_ORDER` 固定了 survey 的组织骨架，`SURVEY_SECTION_CITATION_FLOORS` 则明确了每节至少该有多少引用。这是把“学术写作要求”编译成代码约束的做法。
代码定位：`src/research/graph/nodes/draft.py:16-38`。
设计取舍 / 风险 / 效果：规则会牺牲一些自由度，但可以显著减少标题混乱、引用稀疏和结构漂移。

Q67. `_render_skill_context()` 为什么重要?

简答：因为它把多个辅助工件整理成主模型真正能消费的提示上下文。
展开：如果不做这一步，comparison matrix、outline、section evidence map 就只是落盘 JSON，不会真的进入写作。它是“结构化工件 -> LLM prompt”之间的桥。
代码定位：`src/research/graph/nodes/draft.py:266-341`。
设计取舍 / 风险 / 效果：上下文更长，但信息密度更高，尤其适合 survey 这种长文任务。

Q68. 为什么当前 draft 阶段已经具备一定的“流式可见性”?

简答：因为它会先写 scaffold preview，再持续写 markdown snapshot 到 workspace，前端再通过 SSE 读这些文件变化。
展开：这不是 token 级流式，但已经是 artifact 级流式。用户至少能看到结构、大纲和逐步成形的草稿，而不是一直转圈到最后。
代码定位：`src/research/graph/nodes/draft.py:170-195,631-707`，`src/api/routes/tasks.py:157-220`。
设计取舍 / 风险 / 效果：artifact 级流式比纯 spinner 强很多，但距离 ChatGPT 风格 token 流仍有差距。

Q69. 系统是怎么修复“引用太少、引用过度集中”的?

简答：通过 minimum citation coverage repair 和 section-level citation redistribution。
展开：draft 后处理会根据目标 citation 数量补齐引用，并把被低估的 citation 重新分配到 major sections，避免整篇文章只靠一两篇论文反复支撑。这个设计直接回应了 survey 写作里最常见的引用失衡问题。
代码定位：`src/research/graph/nodes/draft.py:1197-1475`。
设计取舍 / 风险 / 效果：后处理会让文本更“规则化”，但在当前阶段非常必要，因为模型天然容易引用塌缩。

Q70. 当前 draft 阶段最值得诚实承认的限制是什么?

简答：一是非压缩路径里仍有 `cards[:20]` 截断痕迹，二是 artifact 级流式还不是 token 级实时写作，三是引用修复仍可能不够 aggressive。
展开：这些限制不是架构方向错，而是实现仍在进化。面试时坦诚指出并给出改法，通常比硬说“已经很好”更有说服力。
代码定位：`src/research/graph/nodes/draft.py:344-360,188-195,1197-1475`。
设计取舍 / 风险 / 效果：知道短板在哪里，才能说明你真的理解系统。

八、Review、Grounding 与置信度（71-80）

Q71. `review_node` 的完整职责是什么?

简答：它先做 grounding，再跑 reviewer service，再跑 claim_verification skill，最后产出 `review_feedback` 和 `review_passed`。
展开：这说明 review 不是单一 LLM 打分，而是三层组合：引用解析/claim 验证、规则化 reviewer 检查、skill 化 claim verification。它已经是一个小型质量门系统。
代码定位：`src/research/graph/nodes/review.py:93-228`。
设计取舍 / 风险 / 效果：质量控制更扎实，但耗时和复杂度都明显上升。

Q72. `ground_draft_report()` 为什么值得单独拿出来讲?

简答：因为它把 grounding 流程收敛成了一个可复用的小管线：`resolve_citations -> verify_claims -> format_output`。
展开：review 节点并不直接自己做所有事情，而是先调用这个 grounding helper。这样 report workflow 和 research workflow 在 grounding 语义上更容易对齐。
代码定位：`src/research/services/grounding.py:16-74`。
设计取舍 / 风险 / 效果：把 grounding 单独封装出来是对的，它让质量验证可以复用，也方便后续单独评测。

Q73. `verify_claims` 是怎么把 claim 判成 grounded / partial / ungrounded / abstained 的?

简答：它逐 claim 遍历引用标签，调用 `judge_claim_citation()` 做 citation-level 判断，再聚合出 claim-level overall status。
展开：只要有一个 support 是 `supported`，整体就能到 `grounded`；如果只有 `partial` 则是 `partial`；全是不可验证才 `abstained`；其他情况为 `ungrounded`。这是一个非常清楚的层级化判定规则。
代码定位：`src/graph/nodes/verify_claims.py:7-72`。
设计取舍 / 风险 / 效果：规则透明，便于解释；缺点是仍依赖 citation content 质量，证据源弱时就会被拖低。

Q74. `ReviewerService` 和 claim verification skill 的关系是什么?

简答：前者是系统级 review gate，后者是 skill 化的 claim-grounding 统计补充，两者互补而不是互斥。
展开：`ReviewerService` 关注覆盖、citation reachability、结构重复和质量门；claim verification skill 更偏 claim-level grounding 统计。两者一起用，才能既有结构性反馈，又有 claim 证据画像。
代码定位：`src/research/graph/nodes/review.py:37-56,183-228`，`src/research/services/reviewer.py:54-155`。
设计取舍 / 风险 / 效果：双重检查更稳，但会拉长 review 阶段时长。

Q75. 为什么在没有 `paper_cards` 且没有 `draft_report` 时会直接早退失败?

简答：因为这意味着 search/extract/draft 主链已经明显断掉了，再继续 review 只会制造伪结果。
展开：代码会构造一个显式 failed 的 `ReviewFeedback`，并告诉用户“search or extract likely failed”。这是非常好的工程习惯，因为它避免了空输入下继续生成“看似完整”的错误反馈。
代码定位：`src/research/graph/nodes/review.py:152-181`。
设计取舍 / 风险 / 效果：早退显得严格，但能防止系统在空证据上胡乱评价。

Q76. `ReviewerService` 是如何检查 paper card 质量的?

简答：它会检查 title 是否异常、authors/abstract 覆盖率是否过低，并在 bad ratio 过高时直接打 blocker。
展开：这一步很关键，因为垃圾 paper card 会把后面的引用、写作、review 全部污染。它相当于在 review 阶段补了一个“语料层健康检查”。
代码定位：`src/research/services/reviewer.py:159-256`。
设计取舍 / 风险 / 效果：严格检查会让更多任务卡在 review，但这正说明质量门在发挥作用。

Q77. Reviewer 为什么要关心 citation breadth 和 concentration?

简答：因为综述不是找两篇论文反复引用，而是要体现语料覆盖和证据分布。
展开：`ReviewerService` 在主流程里明确调用 citation breadth / balance 检查，这正是在防“看起来写得挺好，但实际上只靠两三篇论文撑全篇”的问题。这个设计和 survey 的写作要求强相关。
代码定位：`src/research/services/reviewer.py:122-128`，`src/research/graph/nodes/draft.py:1197-1475`。
设计取舍 / 风险 / 效果：这会提高通过门槛，但也能直接提升“像综述”的程度。

Q78. 为什么一份报告的 `rag_score` 很高，但 `review_passed` 仍然可能是 false?

简答：因为 retrieval quality 和 grounded writing quality 是两个不同层级的问题。
展开：高 `rag_score` 说明论文找得更准、全文比例更高，但 final review 看的是 claim 是否真的被当前引用支撑、引用是否足够广、结构是否重复、章节是否均衡。也就是说，检索好不等于成文就一定合格。
代码定位：`src/research/graph/nodes/review.py:119-145,183-228`，`src/graph/nodes/format_output.py:80-109`。
设计取舍 / 风险 / 效果：这类回答能解释为什么“看起来进步了，但还是没过 review”。

Q79. `report_confidence` 为什么有时会是 `low`?

简答：因为它不是主观印象分，而是根据 grounded / partial / ungrounded 比例和 degradation mode 计算出来的。
展开：`format_output()` 会先算 grounding stats，再根据 grounded_ratio、partial_ratio、ungrounded_ratio 得到 `high / limited / low`，然后再与 degradation confidence 取更保守值。只要 ungrounded claim 比例高，confidence 就会被压下去。
代码定位：`src/graph/nodes/format_output.py:65-122`。
设计取舍 / 风险 / 效果：好处是 confidence 可解释；坏处是即使文风不错，只要 grounding 差，指标也会很冷酷。

Q80. `revisions/` 目录为什么是 review 阶段的重要配套?

简答：因为它保留了被 review 拦下来的稿件版本，方便人工复盘和下一轮自动修复。
展开：review 不是只输出一个 JSON feedback，而是把失败时的稿件现场也留住。对于调 prompt、调 citation repair、调 claim verification，这比单纯看最终 `report.md` 有用得多。
代码定位：`src/agent/output_workspace.py:227-258`，`src/research/agents/supervisor.py:57-83`。
设计取舍 / 风险 / 效果：revision 多一点文件没关系，关键是它让失败变得可见和可分析。

九、Workspace、持久化与恢复（81-90）

Q81. 为什么项目强调 “workspace-first layout”?

简答：因为系统希望围绕用户的一次研究会话组织多次 task，而不是把每个 task 当孤立文件夹。
展开：目录结构是 `output/workspaces/<workspace_id>/tasks/<task_id>/...`。这意味着 workspace 是比 task 更高一层的组织单位，适合后续做“回到之前 workspace”“同一主题多次迭代”的体验。
代码定位：`src/agent/output_workspace.py:1-26,107-162`。
设计取舍 / 风险 / 效果：相比老的 `output/<task_id>/`，workspace-first 对真实产品更合理。

Q82. `workspace_id` 为什么要带用户前缀和 UTC 时间戳?

简答：因为它既要可读、可排序，又要足够唯一。
展开：`build_workspace_id()` 用 `user + UTC timestamp + uuid short suffix` 组合，既能看出创建时间，又避免纯随机 UUID 难以肉眼识别。这个细节很适合在面试里展示“产品化意识”。
代码定位：`src/agent/output_workspace.py:49-55`。
设计取舍 / 风险 / 效果：可读 ID 会比纯 UUID 稍长，但对排查和人工浏览更友好。

Q83. `ensure_workspace_root()` 和 `create_workspace()` 分别负责什么?

简答：前者负责 workspace 级 manifest，后者负责 task 级目录和 metadata。
展开：`ensure_workspace_root()` 处理 `workspace.json`，维护 `opened_at`、`updated_at`、`task_ids` 等；`create_workspace()` 进入具体 task 目录，创建 `revisions/` 并写 `metadata.json`。两层职责分明。
代码定位：`src/agent/output_workspace.py:62-99,118-162`。
设计取舍 / 风险 / 效果：分层后逻辑更清晰，也为“一个 workspace 多个 task”提供了自然支撑。

Q84. 节点输出是怎么映射成具体文件的?

简答：通过 `write_node_output()` 按 node_name 做分发，写成 `brief.json`、`rag_result.json`、`draft.md`、`review_feedback.json` 等工件。
展开：这一步非常关键，因为它把抽象的 state patch 变成了前端能读、用户能看、测试能比对的持久工件。draft 和 review 还会顺带写 comparison matrix、skill trace、claim verification 等附加文件。
代码定位：`src/agent/output_workspace.py:285-343`。
设计取舍 / 风险 / 效果：好处是工件体系清晰；坏处是 node_name 改动时，这里也必须同步更新。

Q85. `draft.md`、`draft_report.json` 和 `report.md` 分别代表什么?

简答：`draft_report.json` 是结构化草稿对象，`draft.md` 是中间文本草稿，`report.md` 是最终对外展示文本。
展开：系统优先把 draft 阶段的 markdown 流出来，再在最终完成时写 `report.md`。如果只有结构化对象没有 markdown，也会退而写 `draft_report.json`，保证状态不丢。
代码定位：`src/agent/output_workspace.py:190-224,261-282,310-340`。
设计取舍 / 风险 / 效果：三种产物分别服务不同场景，区分开是对的。

Q86. 为什么 `write_report()` 还要更新时间戳和 manifest?

简答：因为最终 report 写出意味着 task 进入一个新的生命周期节点，不能只写文件不更新元数据。
展开：`write_report()` 会更新 `metadata.json` 的 `completed_at`，并触碰 workspace manifest。这样列表页、恢复逻辑和前端展示才能感知“这项任务现在真的完成了”。
代码定位：`src/agent/output_workspace.py:261-282`。
设计取舍 / 风险 / 效果：多一步元数据维护，换来状态一致性。

Q87. 为什么 `append_revision()` 采用递增编号文件名?

简答：因为 revision 是有顺序语义的，按 `001_...md`、`002_...md` 命名最直观。
展开：它既支持显式 label，也支持从内容自动推断 label。这样 revision 既适合机器处理，也适合人类直接打开浏览。
代码定位：`src/agent/output_workspace.py:227-258`。
设计取舍 / 风险 / 效果：简单文件序号比把修订历史只存数据库更适合当前产品形态。

Q88. 数据库里的 report 记录和 workspace 里的 `report.md` 关系是什么?

简答：数据库是 durable result record，workspace 是可见的文件副本，两者在 task 完成时一起写。
展开：`_run_graph()` 会先调用 `save_task_report()`，再调用 `write_report()`。这意味着系统既能通过 API 从数据库拉最终结果，也能通过文件系统做人工检查和 SSE 预览。
代码定位：`src/api/routes/tasks.py:760-788`。
设计取舍 / 风险 / 效果：双写需要一致性控制，但对可恢复性和用户体验很值。

Q89. 系统是如何支持“进程重启后仍能看到旧任务”的?

简答：通过数据库 snapshot 恢复 task，而不是只靠内存 `_tasks`。
展开：`_get_task_record()` 会在内存 miss 时调用 `load_task_snapshot()`；`list_tasks()` 也会先拉 `list_task_snapshots()`。这说明当前历史任务回看已经不再依赖单进程内存。
代码定位：`src/api/routes/tasks.py:91-107,254-267`。
设计取舍 / 风险 / 效果：这是一个非常关键的产品级改进，因为“刷新页面历史全没了”是很多 Demo 系统的通病。

Q90. 为什么把输出记在 `output/workspaces/` 比“只靠内存记录”更好?

简答：因为用户真正需要的是可回放、可比对、可检查的实际文件，而不是只在浏览器生命周期里存在的状态。
展开：中间产物写到 workspace 后，前端能订阅，开发者能复盘，评测脚本能比对，用户也能回到旧 workspace 看历史内容。这种能力是产品级系统和临时 Demo 的分水岭。
代码定位：`src/agent/output_workspace.py:1-26`，`src/api/routes/tasks.py:157-220`。
设计取舍 / 风险 / 效果：文件 I/O 会更重，但换来的是真实可用的历史记录能力。

十、前端可视化、SSE、测试与后续演进（91-100）

Q91. `useTaskSSE` 管理了哪些核心状态?

简答：节点状态、事件列表、thinking entries、任务总时长、当前阶段、workspaceId、最新报告快照等。
展开：这说明前端不是只拿一个“done/failed”状态，而是在维护一份完整的运行视图模型。特别是 `latestReportMarkdown` 和 `latestReportArtifact`，是 live preview 的关键。
代码定位：`frontend/src/hooks/useTaskSSE.ts:5-38,44-159`。
设计取舍 / 风险 / 效果：状态更全，但 hook 也会更重，需要注意前端状态一致性。

Q92. 前端是怎么把 `report_snapshot` 转成实时文稿预览的?

简答：SSE 收到 `report_snapshot` 后，直接更新 `latestReportMarkdown` 和 `latestReportArtifact`。
展开：也就是说，前端不需要等最终 `/result` 才能看到文本。只要后端 workspace 里某个 Markdown 文件发生变化，前端就能立刻刷新预览。
代码定位：`frontend/src/hooks/useTaskSSE.ts:106-117,123-136`，`src/api/routes/tasks.py:184-199`。
设计取舍 / 风险 / 效果：这是当前 live preview 的基础机制，但它仍然是文件级更新，不是 token 级流式。

Q93. `currentStage`、`taskStatus` 和 `workspaceId` 在前端是怎么保持同步的?

简答：一部分来自初始 `GET /tasks/{id}`，另一部分来自后续 SSE 事件增量更新。
展开：hook 在初始化时先拉 task metadata，随后根据 `node_start`、`node_end`、`status_change` 和事件里的 `workspace_id` 持续覆盖本地状态。这样可以兼顾“第一次打开页面”和“边跑边更”。
代码定位：`frontend/src/hooks/useTaskSSE.ts:57-68,81-137`。
设计取舍 / 风险 / 效果：双来源同步是必须的，但实现上容易出现竞态，需要保持事件字段稳定。

Q94. `GraphView` 为什么要维护 report 和 research 两套静态布局?

简答：因为两条 workflow 的节点集合和拓扑不同，强行用一套图只会让用户更困惑。
展开：`REPORT_LAYOUT` 和 `RESEARCH_LAYOUT` 是两套不同的可视布局，边定义也不同。它不是自动从后端读取图谱，而是前端自己维护一份展示模型。
代码定位：`frontend/src/components/GraphView.tsx:66-120,138-192`。
设计取舍 / 风险 / 效果：静态布局易控、观感稳定，但最大的风险就是后端节点变了而前端没同步。

Q95. 为什么说前端当前存在 graph drift 风险?

简答：因为节点布局和边定义是写死在前端里的，而后端工作流节点是在 Python 里定义的。
展开：当后端新增 `extract_compression` 这种节点时，前端如果不改，显示就会错位或者缺节点。这是典型的“展示层拓扑副本”问题。
代码定位：`src/research/graph/builder.py:78-120`，`frontend/src/components/GraphView.tsx:101-120`。
设计取舍 / 风险 / 效果：短期静态布局足够用，长期最好让前后端共享一份图元数据。

Q96. 为什么当前 live preview 还不够像 ChatGPT 那样自然?

简答：因为当前是 artifact 级刷新，不是 token 级连续流式写作。
展开：后端是写 `draft.md` / `report.md` 文件，SSE 再把整个文件内容读出来，所以用户看到的是“阶段性整块刷新”。要做到 ChatGPT 式体验，需要更细粒度的写作流和更轻的上下文增量。
代码定位：`src/api/routes/tasks.py:157-220`，`frontend/src/hooks/useTaskSSE.ts:106-117`，`src/research/graph/nodes/draft.py:188-195,671-707`。
设计取舍 / 风险 / 效果：当前实现已经比纯 spinner 强很多，但下一步确实应向 token/chunk streaming 演进。

Q97. `SkillOrchestrator` 目前支持哪两种主要用法?

简答：显式 `/skill_id ...` 调用，以及让轻量 LLM 隐式决定是否走 skill chain。
展开：如果用户消息以 `/` 开头，就直接路由到指定 skill；否则先让轻量模型判断要不要用 skill、用哪些 skill、按什么顺序链起来。这个设计已经具备 post-report skills 的雏形。
代码定位：`src/skills/orchestrator.py:59-176,178-247`。
设计取舍 / 风险 / 效果：显式模式稳定、隐式模式更自然，但隐式决策很依赖小模型判断质量。

Q98. 为什么仍然说“报告生成后的主动 skills 交互”是当前缺口?

简答：因为虽然 orchestrator 已经存在，但 `/tasks/{id}/chat` 现在还是普通 report-context chat，没有真正把 skill orchestration 接进主用户路径。
展开：也就是说，能力层已经有了，但产品主入口还没把它接通。当前 chat 更像“继续问报告内容”，还不是“围绕报告主动调 skill 做二次分析”。
代码定位：`src/api/routes/tasks.py:319-366`，`src/skills/orchestrator.py:59-176`，`src/skills/registry.py:331-343,359-525`。
设计取舍 / 风险 / 效果：这是非常典型的“基础能力已就位，产品整合没做完”的状态，面试时最好明确讲出来。

Q99. 为什么仓库要求在跑端到端测试前先执行 `python tests/api/check_env_gpt_models.py`?

简答：因为这个系统依赖多个 OpenAI-compatible model 配置，环境一旦不可调用，e2e 结果就没有解释价值。
展开：脚本会显式读取 `.env`，枚举 GPT 模型配置，实际发请求探活，还可以拉 gateway model 列表做对照。这一步本质上是把“环境问题”和“系统问题”分离开。
代码定位：`AGENTS.md:22-24`，`tests/api/check_env_gpt_models.py:1-220`。
设计取舍 / 风险 / 效果：增加了一步 preflight，但它能避免把模型网关故障误判成工作流 bug。

Q100. 从当前代码出发，最值得继续优化的三件事是什么?

简答：第一是更自然的正文流式输出，第二是把 post-report skills 真接进主聊天路径，第三是继续压缩 citation concentration 和 ungrounded claims。
展开：这三件事正好覆盖 UX、能力整合和结果质量三个方向。它们不是“重写系统”，而是在现有架构上继续打磨最短板的地方。
代码定位：`src/research/graph/nodes/draft.py:188-195,1197-1475`，`src/api/routes/tasks.py:319-366,369-393`，`src/skills/orchestrator.py:59-176`，`src/graph/nodes/format_output.py:80-109`。
设计取舍 / 风险 / 效果：继续优化时，应优先保持现有 `task -> workspace -> SSE -> review` 主链路稳定，不要为了某个局部体验破坏整体一致性。

面试时的使用建议

1. 先用 Q01-Q10 把项目整体讲清楚，再看面试官是往 workflow、RAG、写作还是前端追问。
2. 如果对方偏后端，优先答 Q11-Q20、Q81-Q90、Q99。
3. 如果对方偏 Agent / LangGraph，优先答 Q21-Q40、Q61-Q80。
4. 如果对方偏产品与演示效果，优先答 Q91-Q100，并主动展示 workspace 和 live preview。
5. 真正高水平的回答，不是把 100 题背下来，而是能随时把问题落回具体代码文件和设计取舍。